Le format JSON et AJAX

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Le format JSON **4**

A. Le format JSON 4

B. Exercice : Quiz 8

III - AJAX **10**

A. AJAX 10

B. Exercice : Quiz 14

IV - Essentiel **16**

V - Auto-évaluation **17**

A. Exercice 17

B. Test 18

Solutions des exercices **20**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 heure

Prérequis : les bases de HTML et JavaScript

Environnement de travail : Visual Studio Code

Contexte

Lorsque nous découvrons le langage JavaScript, nous découvrons forcément deux formats utiles pour transporter des données : JSON et AJAX. Vous verrez que les deux sont étroitement liés lorsqu'on veut traiter les données.

JSON est un sous-ensemble de la syntaxe JavaScript pour transporter des données et les « *consommées* ». Il s'agit d'une combinaison de requête (request), intégrer au navigateur, et de JavaScript et HTML DOM pour afficher ou utiliser les données.

De plus, JSON est « *autodescriptif* » et facile à comprendre lorsqu'on analyse (parse) des données. Quant à AJAX, il a cette particularité de lire les données d'un serveur web, après le chargement de la page, mettre à jour une page web sans recharger la page et envoyer des données à un serveur web en arrière-plan.

Tous droits réservés à STUDI - Reproduction interdite

II Le format JSON

A. Le format JSON

Az Définition

JSON signifie JavaScript Object Notation. Il s'agit d'un format texte brut pour stocker et transporter des données légères entre ordinateurs. Il s'agit d'un langage indépendant, car dérivé de la syntaxe du JavaScript object notation. Le code pour lire et générer du JSON existe dans de nombreux langages de programmation.

Une chaîne (string) JSON

Exemple

```
'{"name" :"Charles", "age":30, "car":null}'
```

C'est un exemple d'objet avec 3 propriétés :

- name
- age
- car

Chaque propriété a sa valeur. Si on parse (analyse) la chaîne JSON avec un programme JavaScript, on peut accéder aux données en tant qu'objet :

```
let jsonString = '{"name":"John", "age":30}';
let obj = JSON.parse(jsonString);
let personName = obj.name;
let personAge = obj.age;
console.log(personName);
console.log(personAge);
```

Pourquoi utiliser JSON ? Le format JSON est syntaxiquement similaire au code de création d'objets JavaScript. Pour cette raison, un programme JavaScript peut facilement convertir des données JSON en objets JavaScript.

Depuis que le format est du texte brut uniquement, les données JSON peuvent facilement être envoyées entre ordinateurs et utilisées par n'importe quel langage. JavaScript a une fonction intégrée pour convertir les chaînes JSON en objets JavaScript, la fonction JSON.parse(). JavaScript a également une fonction pour convertir un objet en chaîne JSON, la fonction JSON.stringify().

Stocker des données

Lors du stockage des données, les données doivent être dans un certain format, et, quel que soit l'endroit où vous choisissez de les stocker, le texte est toujours l'un des formats légaux. JSON rend possible de stocker des objets JavaScript sous forme de texte.

Tous droits réservés à STUDI - Reproduction interdite

Points importants à retenir ⚠ Attention

Nous pouvons recevoir du texte pur depuis un serveur et l'utiliser comme un objet JavaScript, envoyer un objet JavaScript à un serveur au format texte pur et travailler avec des données en tant qu'objets JavaScript, sans analyse ni traductions complexes.

La syntaxe JSON

La syntaxe JSON est un sous-ensemble de la syntaxe JavaScript. Voici les 4 règles de la syntaxe :

- Les données sont dans des paires nom/Valeur.
- Les données sont séparées par des virgules.
- Les accolades contiennent des objets.
- Les crochets contiennent des tableaux.

Une paire nom/valeur se compose d'un nom de champ (entre guillemets doubles), suivi de deux-points, suivi d'une valeur. Le format JSON est presque identique aux objets JavaScript. Les clés doivent être des chaînes, écrites avec des guillemets :

Une chaîne au format JSON :

```
1 {"name" :"Charles"}
```

Un objet au langage JavaScript :

```
1 {name:"Charles"}
```

Le type de fichier pour les fichiers JSON est « *.json* ».

Les valeurs JSON

Dans JSON, les valeurs doivent être l'un des types de données suivantes :

- Une chaîne (string)
- Un nombre (number)
- Un objet (object)
- Un tableau (array)
- Un booléen (boolean - true/false)
- Null

Dans JavaScript, les valeurs peuvent être toutes les valeurs ci-dessus, plus toute autre expression JavaScript valide, y compris une fonction (function), une date (date) et undefined.

JSON vs XML

JSON et XML peuvent être utilisés pour recevoir des données d'un serveur web. Les deux présentent des spécificités non négligeables pour rédiger et transporter les données de la façon la plus claire qui soit.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Les exemples JSON et XML suivants définissent tous deux un objet "*employees*", avec un tableau de 3 employés.

Objet avec tableau au format JSON :

```
{"employees":[
  { "firstName":"Charles", "lastName":"Dupont" },
  { "firstName":"Marie", "lastName":"Leclerc" },
  { "firstName":"Pierre", "lastName":"Lefranc" }
]}
```

Objet au format XML :

```
<employees>
  <employee>
    <firstName>Charles</firstName> <lastName>Dupont</lastName>
  </employee>
  <employee>
    <firstName>Marie</firstName> <lastName>Leclerc</lastName>
  </employee>
  <employee>
    <firstName>Pierre</firstName> <lastName>Lefranc</lastName>
  </employee>
</employees>
```

Ces deux exemples mettent en avant que le JSON est comme le XML parce que « *autodescriptifs* », hiérarchiques (valeurs dans des valeurs), peuvent être analysés et utilisés par de nombreux langages de programmation. Ils peuvent également être récupérés avec un XMLHttpRequest.

Aussi, ces 2 exemples démontrent que le JSON est différent du XML parce que JSON n'utilise pas de balise de fin, est plus court, plus rapide à lire et écrire, et peut utiliser des tableaux.

Surtout, la plus grande différence est que XML doit être analysé (parse) avec un analyseur (parser) XML. JSON peut être analysé par une fonction JavaScript standard.

Points importants à retenir

⚠ Attention

XML est plus difficile à analyser qu'un JSON. JSON est analysé dans un objet JavaScript prêt à l'emploi.

Les types de données JSON

Les chaînes (strings) dans JSON doivent être écrites avec des guillemets doubles. Les nombres (numbers) dans JSON doivent être un entier (integer) ou une virgule flottante (floating point). Les valeurs dans JSON peuvent être des objets (objects), des tableaux (arrays), des booléens (boolean) true/false, ou peuvent être null.

number JSON : `{"age":30}`

object JSON :

```
{
"employees":{"name":"Charles", "age":30, "city":"Paris"}
}
```


Tous droits réservés à STUDI - Reproduction interdite

array JSON :

```
{
"employees":["Charles", "Marie", "Pierre"]
}
```

boolean : `{"rain":true}`

null JSON : `{"middlename":null}`

> ⚠ Attention
>
> Les objets en tant que valeurs dans JSON doivent suivre la syntaxe JSON.

JSON.parse()/JSON.stringify()/Stocker des données

Une utilisation courante de JSON consiste à échanger des données vers/depuis un serveur web. Lors de la réception de données d'un serveur web, les données sont toujours une chaîne. Analysez les données avec JSON.parse() et les données deviennent un objet JavaScript. Aussi, convertissez un objet JavaScript en chaîne avec JSON.stringify().

Lors du stockage des données, les données doivent être dans un certain format, et, quel que soit l'endroit où vous choisissez de les stocker, le texte est toujours l'un des formats légaux. JSON permet de stocker des objets JavaScript sous forme de texte.

Nous allons voir ces trois points dans cette vidéo.

Nous venons de voir comment analyser des données JSON afin qu'ils deviennent des objets JavaScript avec JSON.parse(). Aussi, nous venons de voir la conversion des objets JavaScript en chaîne JSON avec JSON.stringify(). Pour finir, le stockage et la récupération des données avec localStorage.

> ⚠ Attention
>
> **Exceptions**
>
> Les objets date ne sont pas autorisés dans JSON. Si vous devez inclure une date, écrivez-la sous forme de chaîne. Vous pouvez le reconvertir en objet date ultérieurement.
>
> Aussi, vous devez éviter d'utiliser les fonctions dans JSON, les fonctions perdront leur portée et vous devrez utiliser eval() pour les reconvertir en fonctions.

Serveur JSON

Vous pouvez faire une requête JSON au serveur en utilisant une requête Ajax. Tant que la réponse du serveur est écrite au format JSON, vous pouvez analyser la chaîne dans un objet JavaScript. Il faut utiliser XMLHttpRequest pour obtenir des données du serveur :

```
<!DOCTYPE html>
<html>
  <body>
    <p id="demo"></p>

  <script>
    const xmlhttp = new XMLHttpRequest();
    xmlhttp.onload = function() {
        const myObj = JSON.parse(this.responseText);
        document.getElementById("demo").innerHTML = myObj.name;
    }
    xmlhttp.open("GET",  "file.json");
```

```
      xmlhttp.send();
    </script>

    </body>
  </html>
```

Lors de l'utilisation de JSON.parse() sur JSON dérivé d'un tableau, la méthode renverra un tableau JavaScript au lieu d'un objet JavaScript.

```
<!DOCTYPE html>
<html>
  <body>
    <p id="demo"></p>

    <script>
      const xmlhttp = new XMLHttpRequest();
      xmlhttp.onload = function() {
        const myArr = JSON.parse(this.responseText);
        document.getElementById("demo").innerHTML = JSON.stringify(myArr, null, 2);
      }
      xmlhttp.open("GET", "car.json", true);
      xmlhttp.send();
    </script>
  </body>
</html>
```

B. Exercice : Quiz

[solution n°1 p. 20]

Question 1

Que signifie JSON ?

- Jirascript object notation
- JavaScript Object Notation
- Java object notation

Question 2

Quelles sont les 4 règles de syntaxe JSON ? Plusieurs réponses sont possibles :

- **A** Les données sont dans des paires nom/valeur
- **B** Les données sont séparées par des virgules
- **C** Les puces contiennent des numéros
- **D** Les accolades contiennent des objets
- **E** Les crochets contiennent des tableaux

Question 3

Quelle est la fonction intégrée à JavaScript pour convertir un objet en chaîne JSON ?

Tous droits réservés à STUDI - Reproduction interdite

string()

stringify()

JSON.stringify()

Question 4

Quelle est la fonction intégrée à JavaScript pour convertir une chaîne JSON en objet JavaScript ?

parse()

JSON.parse()

JSON()

Question 5

```
const xmlhttp = new XMLHttpRequest();
xmlhttp.onload = function() {
  const myObj = JSON.parse(this.responseText);
  document.getElementById(« demo »).innerHTML =
  myObj[0];
  };
xmlhttp.open("GET", "car.json", true);
xmlhttp.send();
```

Lorsque je fais cette requête AJAX avec le JSON : ["*Ford*", "*Renault*", “*Audi*”,"*Fiat*"]. Quelle sera la réponse ?

“Ford”

“Renault”

“Audi”

“Fiat”

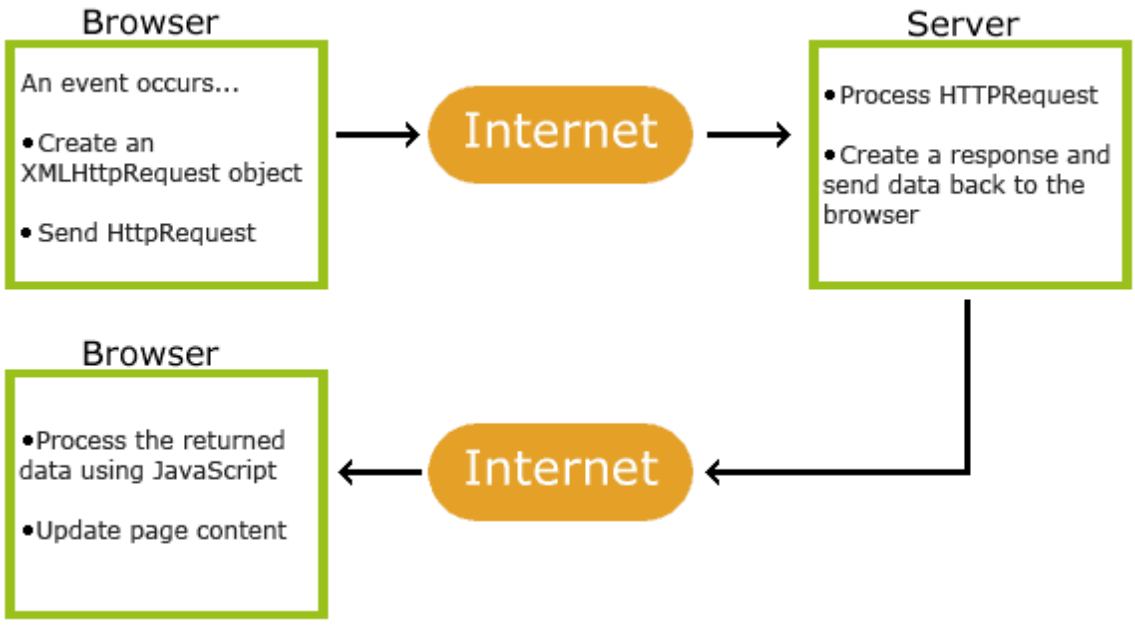
Tous droits réservés à STUDI - Reproduction interdite

III AJAX

A. AJAX

Az Définition

AJAX signifie **A**synchronous **J**avascript **A**nd **X**ML. Il ne s'agit pas d'un langage de programmation. C'est une combinaison de requêtes (request) intégrées au navigateur et de JavaScript et HTML DOM pour afficher ou utiliser les données. L'acronyme peut prêter à confusion. Malgré son nom, AJAX peut transporter des données XML, mais également d'autres formats de texte comme JSON ou txt.



Source : w3schools[1]

Comment fonctionne AJAX ?

Pour répondre à cette question, voici un schéma.

Un événement a lieu dans le navigateur, une requête HTTP est créée et envoyée au serveur. Le serveur traite la requête HTTP, génère une réponse et envoie les données en retour au navigateur. Les données envoyées par le serveur sont lues par JavaScript et la page est mise à jour.

XMLHttp

La pierre angulaire d'AJAX est l'objet XMLHttpRequest. Tous les navigateurs modernes le prennent en charge. Il peut être utilisé pour échanger des données avec un serveur en arrière-plan. Cela signifie qu'il est possible de mettre à jour des parties d'une page web, sans recharger la page.

Aussi, il faut savoir que pour des raisons de sécurité les navigateurs n'autorisent pas l'accès entre les domaines. Cela signifie que la page web et le fichier XML, ou texte, qu'elle tente de charger doivent se trouver sur le même serveur. Si vous souhaitez charger les fichiers XML, ils doivent se trouver sur votre propre serveur.

1. https://www.w3schools.com/

Tous droits réservés à STUDI - Reproduction interdite

Également, l'objet XMLHttpRequest peut avoir différentes méthodes. Voici un tableau pour résumer ces méthodes :

Méthode	Description
new XMLHttpRequest()	Crée un nouvel objet XMLHttpRequest
abort()	Annule la demande en cours
getAllResponseHeaders()	Renvoie les informations d'en-tête
getResponseHeader()	Renvoie une information spécifique d'en-tête
open(method, url, async, user, psw)	Spécifie la demande method : la requête de type GET ou POST url : l'emplacement du fichier async : true(asynchrone) ou false(synchrone) user : nom d'utilisateur facultatif psw : mot de passe facultatif
send()	Envoie la requête au serveur Utilisé pour les requêtes GET
send(string)	Envoie la requête au serveur Utilisé pour les requêtes POST
setRequestHeader()	Ajoute une étiquette/valeur à l'en-tête à envoyer

L'objet XMLHttpRequest a également différentes propriétés. Voici un tableau pour les résumer :

Propriété	Description
onreadystatechange	Définit une fonction à appeler lorsque la propriété readyState change
readyState	Contient le statut de XMLHttpRequest 0 : requête non initialisée 1 : connexion au serveur établie 2 : requête reçue 3 : traitement en cours 4 : requête achevée et la réponse est prête
responseText	Renvoie les données sous forme de chaîne
responseXML	Renvoie les données sous forme de données XML

Tous droits réservés à STUDI - Reproduction interdite

Propriété	Description
status	Renvoie le numéro d'état d'une requête 200 : « *OK* » 403 : « *Forbidden* » 404 : « *Not Found* » Pour une liste complète, cliquez sur ce lien : HTTP response status codes[2]
statusText	Renvoie le texte d'état (par exemple « *OK* » ou « *Not Found* »)

Envoyer une requête au serveur avec GET ou POST

Pour envoyer une requête à un serveur, nous utilisons les méthodes open() et send() de l'objet XMLHttpRequest, en utilisant soit GET ou POST. Il faut savoir que GET est simple et plus rapide que POST, et peut être utilisé dans la plupart des cas. Pour autant, il faut toujours utiliser POST lorsqu'un fichier en cache n'est pas une option (mettre à jour un fichier ou une base de données sur le serveur), envoyer une grande quantité de données au serveur (POST n'a pas de limitation de taille), et envoyer des entrées utilisateur (qui peuvent contenir des caractères inconnus). POST est plus robuste et sécurisé que GET.

Effectuer une requête au serveur avec POST

Exemple

Dans cet exemple, à partir des éléments que nous avons vus au-dessus, vous devez effectuer une requête POST avec l'objet XMLHttpRequest. Dans ce cas, vous allez créer un fichier .txt et le "*consommer*". Les données du fichier .txt devront apparaître en cliquant sur un bouton. Aussi, il faudra que la requête soit asynchrone et ajouter la fonction "*onreadystatechange*". Pour cela, il faudra vous référer au tableau sur les propriétés de l'objet XMLHttpRequest.

Explication :

1 - Nous créons un fichier texte.

```
<p>Salut, je m'appelle Charles, j'ai 34 ans et j'habite Paris.</p>
<p>Aussi, j'apprends AJAX.</p>
```

2 - Nous créons notre fichier HTML avec notre bouton.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>Le format JSON et AJAX</title>
  <link href="style.css" rel="stylesheet" type="text/css" />
</head>
<body>

<div id="demo">
<h1>L'objet XMLHttpRequest</h1>
<button type="button" onclick="loadDoc()">Voir le texte</button>
</div>

```

2. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Tous droits réservés à STUDI - Reproduction interdite

```
</body>
</html>
```

3 - Ensuite, nous créons notre script.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>Le format JSON et AJAX</title>
  <link href="style.css" rel="stylesheet" type="text/css" />
</head>
<body>

<div id="demo">
  <h1>L'objet XMLHttpRequest</h1>
  <button type="button" onclick="loadDoc()">Voir le texte</button>
</div>

<script>
  // Création de la fonction
  function loadDoc() {
    const xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
      if (this.readyState == 4 && this.status == 200) {
        document.getElementById("demo").innerHTML = this.responseText;
      }
    };
    xhttp.open("GET", "demo.txt", true);
    xhttp.send();
  }
</script>

</body>
</html>
```

4 - Voici le résultat dans le navigateur.

Salut, je m'appelle Charles, j'ai 34 ans et j'habite Paris.

Aussi, j'apprends AJAX.

⚠ Attention

Les requêtes XMLHttpRequest synchrones (async = false) ne sont pas recommandées parce que JavaScript va s'arrêter jusqu'à ce que la réponse du serveur soit prête. Si le serveur est occupé ou lent, l'application "*freeze*" s'arrête. C'est pour cette raison que cette requête est en train d'être supprimée de la norme web.

⚠ Attention

Les versions d'Internet Explorer antérieures à IE 10 ne prennent pas en charge l'objet XMLHttpRequest, qui est la base des requêtes AJAX.

Tous droits réservés à STUDI - Reproduction interdite

Réponse du serveur

Dans cette vidéo, nous allons voir comment obtenir une réponse du serveur à travers différents exemples.

Le fichier XML

AJAX peut être utilisé de façon interactive avec les fichiers XML. C'est ce que nous allons voir dans cette vidéo.

Nous avons vu dans cette vidéo comment afficher les données d'un fichier .xml sous forme de tableau. Ainsi, les possibilités pour créer des applications interactives en utilisant XML, HTTP, DOM et JavaScript sont vastes.

B. Exercice : Quiz

[solution n°2 p. 22]

Question 1

Que signifie AJAX ?

- **A**synchronous **J**avascript **A**nd **X**ML
- **S**ynchronous **J**avascript **A**nd **X**ML
- **A**sync **J**avascript **A**nd **X**ML

Question 2

Quels types de données AJAX peut-il transporter ? Plusieurs réponses sont possibles.

- **A** .xml
- **B** .json
- **C** .xls
- **D** .txt

Question 3

Quelle méthode est utilisée pour les requêtes GET envoyées au serveur ?

- abort()
- send()
- send(string)

Question 4

Quel est le type de requête qu'il faut toujours utiliser dans les méthodes open() et send() ?

Tous droits réservés à STUDI - Reproduction interdite

POST

GET

XMLHttpRequest

Question 5

Quelle est la fonction de la propriété onreadystatechange dans l'objet XMLHttpRequest ?

Elle définit une fonction appelée chaque fois que l'état de la requête change

Elle définit une fonction appelée lorsque la requête est en cours d'envoi

Elle définit une fonction appelée lorsqu'une erreur survient lors de l'envoi de la requête

Elle définit une fonction appelée lorsque la requête est terminée et que la réponse est prête à être traitée

Tous droits réservés à STUDI - Reproduction interdite

IV Essentiel

L'utilisation d'AJAX (Asynchronous JavaScript and XML) est une technique de programmation qui a révolutionné le développement des applications web modernes. Cette technique permet de mettre à jour dynamiquement les contenus d'une page web sans avoir besoin de la recharger complètement, offrant ainsi une expérience utilisateur plus réactive et plus fluide.

L'objet XMLHttpRequest en JavaScript, qui permet de créer des requêtes HTTP asynchrones depuis une page web vers un serveur web, a rendu possible l'utilisation d'AJAX. Cela signifie que la page web peut envoyer des requêtes au serveur sans que l'utilisateur ait à quitter la page ou à attendre une nouvelle page pour voir les résultats.

En combinant AJAX avec les technologies du web, telles que HTML, CSS et JavaScript, les développeurs peuvent créer des interfaces utilisateur riches et dynamiques, sans sacrifier la performance ou la qualité de l'expérience utilisateur.

L'utilisation d'AJAX est courante dans de nombreux domaines d'application, tels que les réseaux sociaux, les applications de messagerie, les outils de collaboration, les jeux en ligne, les applications de gestion de contenu et bien d'autres. Dans ces applications, AJAX est utilisé pour charger des contenus dynamiques, envoyer des messages, effectuer des recherches, soumettre des formulaires et mettre à jour les données de manière asynchrone.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Votre société de développement vous confie une "*user story*" (une fonctionnalité) pour l'un de ses clients. L'utilisateur doit pouvoir voir une liste de villes avec le code postal où se trouvent des dépôts en denrées alimentaires dans son département, sous forme de tableau. Ensuite, lorsqu'il clique sur une ville, cela affiche des informations sur l'adresse, date et horaire des dépôts ouverts au public. Il vous faudra consommer le fichier XML avec une requête AJAX.

Fichier XML donné :

```xml
<DEPARTMENT>
    <CITY>
        <NAME>Vernon</NAME>
        <ZIP>27200</ZIP>
        <ADRESS>30 rue des carreaux</ADRESS>
        <DAY>03/05/23/</DAY>
        <HOUR>14H00 - 18h00</HOUR>
    </CITY>
    <CITY>
        <NAME>Gaillon</NAME>
        <ZIP>27600</ZIP>
        <ADRESS> 10 rue des joyeux</ADRESS>
        <DAY>13/05/23/</DAY>
        <HOUR>14H00 - 18h00</HOUR>
    </CITY>
    <CITY>
        <NAME>Louviers</NAME>
        <ZIP>27350</ZIP>
        <ADRESS>60 avenue des vaches</ADRESS>
        <DAY>08/05/23/</DAY>
        <HOUR>14H00 - 18h00</HOUR>
    </CITY>
    <CITY>
        <NAME>Gisors</NAME>
        <ZIP>27420</ZIP>
        <ADRESS>Rue Victor Leray</ADRESS>
        <DAY>30/05/23/</DAY>
        <HOUR>14H00 - 18h00</HOUR>
    </CITY>
    <CITY>
        <NAME>Aubevoye</NAME>
        <ZIP>27360</ZIP>
        <ADRESS>Le Clos Maurice</ADRESS>
        <DAY>23/05/23/</DAY>
        <HOUR>14H00 - 18h00</HOUR>
    </CITY>
</DEPARTMENT>
```

Fichier HTML donné :

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Exercice</title>
  </head>
  <style>
```

Tous droits réservés à STUDI - Reproduction interdite

```
    table,
    th,
    td {
      border: 1px solid black;
      border-collapse: collapse;
    }
    th,
    td {
      padding: 5px;
    }
  </style>
  <body>
    <h1>
      Rechercher un dépôt dans votre ville
    </h1>

    <br /><br />
    <p id="showCity"></p>
    <table id="demo"></table>
</body>
</html>
```

Question 1

[solution n°3 p. 23]

Créer la requête AJAX et ajouter une fonction pour afficher les villes avec leurs codes postaux sous forme de tableau.

Question 2

[solution n°4 p. 24]

Créer une fonction pour afficher les informations pour chaque ville lorsqu'on clique dessus.

B. Test

Exercice 1 : Quiz

[solution n°5 p. 24]

Question 1

AJAX ne peut fonctionner qu'avec des données au format XML.

- Vrai
- Faux

Question 2

Les requêtes AJAX ne peuvent pas être annulées une fois lancées.

- Vrai
- Faux

Question 3

Les requêtes AJAX sont toujours synchrones.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Les requêtes AJAX ne peuvent pas être utilisées pour envoyer des fichiers au serveur.

- Vrai
- Faux

Question 5

Les requêtes AJAX ne sont pas compatibles avec les anciennes versions d'Internet Explorer.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 8]

Question 1

Que signifie JSON ?

Jirascript object notation

✓ JavaScript Object Notation

Java object notation

L'acronyme JSON signifie « *JavaScript Object Notation* ».

Question 2

Quelles sont les 4 règles de syntaxe JSON ? Plusieurs réponses sont possibles :

A Les données sont dans des paires nom/valeur

B Les données sont séparées par des virgules

C Les puces contiennent des numéros

D Les accolades contiennent des objets

E Les crochets contiennent des tableaux

La syntaxe JSON respecte les 4 règles suivantes : les données sont présentées par paires nom/valeur et doivent être séparées par des virgules ; les accolades doivent contenir des objets ; enfin, les crochets doivent contenir des tableaux.

Question 3

Quelle est la fonction intégrée à JavaScript pour convertir un objet en chaîne JSON ?

string()

stringify()

Tous droits réservés à STUDI - Reproduction interdite

✓ JSON.stringify()

La fonction JSON.stringify() est intégrée à JavaScript et permet de convertir un objet en chaîne JSON.

Question 4

Quelle est la fonction intégrée à JavaScript pour convertir une chaîne JSON en objet JavaScript ?

parse()

✓ JSON.parse()

JSON()

La fonction JSON.parse() est intégrée à JavaScript et permet de convertir une chaîne JSON en objet JavaScript.

Question 5

```
1 const xmlhttp = new XMLHttpRequest();
2 xmlhttp.onload = function() {
3   const myObj = JSON.parse(this.responseText);
4   document.getElementById(« demo »).innerHTML =
5   myObj[0];
6   };
7 xmlhttp.open("GET", "car.json", true);
8 xmlhttp.send();
```

Lorsque je fais cette requête AJAX avec le JSON : ["*Ford*", "*Renault*", *“Audi”*,"*Fiat*"]. Quelle sera la réponse ?

✓ “Ford”

“Renault”

“Audi”

“Fiat”

Le fichier JSON est un tableau. Lorsque nous parcourons le premier élément du tableau avec la fonction JSON.parse() dans notre const myObj, nous obtenons "*Ford*".

Tous droits réservés à STUDI - Reproduction interdite

Solution n°2

[exercice p. 14]

Question 1

Que signifie AJAX ?

- ✓ **A**synchronous **J**avascript **A**nd **X**ML
- **S**ynchronous **J**avascript **A**nd **X**ML
- **A**sync **J**avascript **A**nd **X**ML

AJAX (Asynchronous JavaScript and XML) est une technique de programmation qui permet de faire des échanges de données entre un serveur web et une page web, sans avoir à recharger complètement la page.

Question 2

Quels types de données AJAX peut-il transporter ? Plusieurs réponses sont possibles.

- **A** .xml
- **B** .json
- **C** .xls
- **D** .txt

Malgré son nom, AJAX peut transporter des données .xml, .json, .txt.

Question 3

Quelle méthode est utilisée pour les requêtes GET envoyées au serveur ?

- abort()
- ✓ send()
- send(string)

La fonction send() est une méthode de l'objet XMLHttpRequest en JavaScript, utilisée pour envoyer une requête HTTP asynchrone au serveur web.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quel est le type de requête qu'il faut toujours utiliser dans les méthodes open() et send() ?

- ✓ POST
- GET
- XMLHttpRequest

La requête POST est plus robuste et sécurisée que GET.

Question 5

Quelle est la fonction de la propriété onreadystatechange dans l'objet XMLHttpRequest ?

- ✓ Elle définit une fonction appelée chaque fois que l'état de la requête change
- Elle définit une fonction appelée lorsque la requête est en cours d'envoi
- Elle définit une fonction appelée lorsqu'une erreur survient lors de l'envoi de la requête
- Elle définit une fonction appelée lorsque la requête est terminée et que la réponse est prête à être traitée

La propriété onreadystatechange permet de spécifier une fonction de rappel qui sera appelée chaque fois que l'état de la requête change. La fonction de rappel peut être utilisée pour mettre à jour dynamiquement les contenus de la page web en fonction de la réponse du serveur.

[exercice p. 18]

Solution n°3

```
<script>
      //Notre requête AJAX
      const xhttp = new XMLHttpRequest();
      let city;
      xhttp.onload = function() {
        const xmlDoc = xhttp.responseXML;
        city = xmlDoc.getElementsByTagName("CITY");
        loadCity();
      };
      xhttp.open("GET", "city.xml");
      xhttp.send();

//Notre fonction pour analyser notre XML et afficher sous forme de tableau
function loadCity() {
        let table = "<tr><th>Ville</th><th>Code postal</th></tr>";
        for (let i = 0; i < city.length; i++) {
```

Tous droits réservés à STUDI - Reproduction interdite

```
          table += "<tr onclick='displayCity(" + i + ")'><td>";
          table += city[i].getElementsByTagName("NAME")[0].childNodes[0]
            .nodeValue;
          table += "</td><td>";
          table += city[i].getElementsByTagName("ZIP")[0].childNodes[0]
            .nodeValue;
          table += "</td></tr>";
        }
        document.getElementById("demo").innerHTML = table;
      }

</script>
```

[exercice p. 18]

Solution n°4

```
<script>
//Notre fonction pour afficher les informations pour chaque ville
function displayCity(i) {
        document.getElementById("showCity").innerHTML =
          "Dépôt : " +
          city[i].getElementsByTagName("NAME")[0].childNodes[0].nodeValue +
          "<br>Code postal : " +
          city[i].getElementsByTagName("ZIP")[0].childNodes[0].nodeValue +
          "<br>Adresse : " +
          city[i].getElementsByTagName("ADRESS")[0].childNodes[0].nodeValue +
          "<br>Jour : " +
          city[i].getElementsByTagName("DAY")[0].childNodes[0].nodeValue +
          "<br>Horaire : " +
          city[i].getElementsByTagName("HOUR")[0].childNodes[0].nodeValue;
      }
</script>
```

Solution n°5

[exercice p. 18]

Question 1

AJAX ne peut fonctionner qu'avec des données au format XML.

Vrai

✓ Faux

Bien que l'acronyme AJAX contienne « *XML* », AJAX peut fonctionner avec différents formats de données, tels que JSON, HTML et texte brut. JSON est d'ailleurs devenu le format de données le plus couramment utilisé dans les applications AJAX en raison de sa simplicité et de sa légèreté.

Question 2

Les requêtes AJAX ne peuvent pas être annulées une fois lancées.

Tous droits réservés à STUDI - Reproduction interdite

Vrai

✓ Faux

Il est possible d'annuler une requête AJAX en utilisant la méthode abort() sur l'objet XMLHttpRequest. Cela permet d'arrêter une requête en cours d'exécution, par exemple si l'utilisateur souhaite interrompre le processus.

Question 3

Les requêtes AJAX sont toujours synchrones.

Vrai

✓ Faux

L'une des principales caractéristiques des requêtes AJAX est leur capacité à être asynchrones. Les requêtes asynchrones permettent d'exécuter plusieurs tâches simultanément sans bloquer l'exécution du reste du code. Bien qu'il soit possible de rendre une requête AJAX synchrone, cela n'est généralement pas recommandé, car cela peut bloquer l'interface utilisateur.

Question 4

Les requêtes AJAX ne peuvent pas être utilisées pour envoyer des fichiers au serveur.

Vrai

✓ Faux

Les requêtes AJAX peuvent être utilisées pour envoyer des fichiers au serveur en utilisant l'objet FormData. FormData est une interface qui permet de construire facilement un ensemble de paires clé-valeur représentant les données du formulaire. Il peut être utilisé avec XMLHttpRequest pour envoyer des fichiers via AJAX, offrant ainsi une expérience utilisateur plus fluide et interactive lors du téléchargement de fichiers.

Question 5

Les requêtes AJAX ne sont pas compatibles avec les anciennes versions d'Internet Explorer.

✓ Vrai

Faux

Tous droits réservés à STUDI - Reproduction interdite

Les versions d'Internet Explorer plus anciennes que IE 10 ne prennent pas en charge l'objet XMLHttpRequest, qui est la base des requêtes AJAX. Pour assurer la compatibilité avec ces anciennes versions d'IE, les développeurs devaient utiliser un objet ActiveXObject spécifique à IE. Toutefois, cette approche est de moins en moins courante, car les anciennes versions d'IE sont désormais obsolètes et la plupart des développeurs se concentrent sur les navigateurs modernes qui prennent en charge les fonctionnalités AJAX standard.

Tous droits réservés à STUDI - Reproduction interdite